Worker isolation: test results (branch fix/worker-isolation, base 35884ee)

New tests: server/lib/job-isolation.test.ts (9/9 pass, exit 0, ~36 s)

Test	Result
Busy child (3 s CPU)	parent max event-loop lag 6.2 ms, wall 3830 ms
Cooperative cancel	resolved “Cancelled” after 2527 ms
Deadline / revoke, child ignores SIGTERM + grandchild	rejected after 6672 ms “Job watchdog timeout: … signal=SIGKILL … after re-voke:job_watchdog_timeout”; child and grandchild gone
Child crash holding LLM permit	“Isolated job child exited unexpectedly (code=1 …)”, retriable; permit released
Permit brokering	max LLM 8, max browser 1, all released
Usage forwarding	rows land in parent buffer, child buffer 0, child PG_POOL_MAX=4
ProviderScoringError	rebuilt as real class (quota_exhausted, deep-seek)
Real runAnalysisPipeline, 40 docs (stub PG/providers)	success, 36 measures, 168 provider calls, 1 fenced write, wall 14716 ms, parent max lag 16.9 ms
childPgBudget()	4

Log: evidence/job-isolation.test.log

Independent re-run (2026-10-11T19:22Z, verified by hand, exit 0)

Re-executed `DATABASE_URL=postgres://stub:stub@127.0.0.1:1/stub npx tsx server/lib/job-isola-tion.test.ts`:
9/9 pass, 0 fail, duration 32.4 s. Busy-child parent max lag **1.6 ms**; full 36-measure real pipeline parent max lag **39.4 ms** (run-to-run variance vs the 16.9 ms above — both well under the 100 ms responsiveness threshold, and orders of magnitude below the seconds-scale production stall). Dead-line
path: child ignoring SIGTERM was SIGKILLed with its process group after the 1 s grace, watchdog rejec-

ted
after 6529 ms, grandchild reaped.

Independent re-run (2026-10-11T19:38Z, verified by hand, exit 0)

Re-executed the same command: `# pass 9 # fail 0`, duration 32.3 s. Full 36-measure real pipeline in child: success, 36 measures, 168 provider calls, 1 fenced write lock, wall 11,869 ms, **parent max lag 12.2 ms**.
Three independent runs now bracket the parent-responsiveness figure at 12.2 / 16.9 / 39.4 ms — run-to-run
variance under system load, every value far under the 100 ms threshold and orders of magnitude below the
seconds-scale production stall this fix targets.

Independent no-regression re-run on every module the commit touches (2026-10-11T19:25Z, by hand)

lifecycle-fence.unit 42/42 · scoring-cancellation.unit 60/60 · credit-breaker 22/22 · transport-abort.unit 55/55 · llm-usage 20/20 · adaptive-concurrency 13/13 — all pass, 0 fail (221 tests incl. the 9 isolation tests). No regressions in any changed module.

Full suite (each test file run with tsx, stub DATABASE_URL)

- Baseline 35884ee: 97 files, 94 pass, 3 fail.
- Branch: 98 files, 95 pass, 3 fail: the same 3 pre-existing failures (corpus-replay.test.ts, lifecycle-fence.db.test.ts needs REDIS_URL, r6-discovery.test.ts vitest missing). No regressions; the only extra file is job-isolation.test.ts (pass).

tsc

- Baseline 61 errors, branch 62. The one new error: TS7016 (no pdf-parse types) in server/scripts/profile-worker-blocking.ts, the same pre-existing error class as processor.ts(4,22). No new errors in runtime files.

Not verified

- No runs against real Postgres/Redis/Railway. WORKER_ISOLATION_ENABLED defaults to off, so staging is still needed.
- Expected costs: ~200 MB and ~1.7-1.9 s cold start per child, up to 4 Chromium instances per replica, DB cap of 4 per job, BatchFetchState merged only at job end, credit-breaker/provider pause state kept per child.